

Nikto — Study Guide

Nikto is a lightweight, fast, open-source **web server scanner** — narrower in scope than Nessus/OpenVAS (web-only, not general infrastructure), but it's often the very first tool run against any discovered HTTP/HTTPS service because it's fast and immediately surfaces low-hanging fruit.

1. What Nikto Checks For

Nikto tests a web server against a database of thousands of known issues:

- Dangerous/outdated files and CGI scripts
- Server misconfigurations
- Outdated server software versions with known vulnerabilities
- Default files/directories left on the server
- Insecure HTTP headers
- Presence of common admin interfaces

It's explicitly **not** a deep application-logic scanner (that's ZAP/Burp territory) — think of it as a fast, broad "known bad stuff" checklist for the web server itself.

2. Basic Usage

```
nikto -h http://target
nikto -h https://target -ssl
nikto -h target -p80,443,8080 # multiple ports
```

3. Key Flags

Flag	Meaning
<code>-h</code>	Target host
<code>-p</code>	Port(s)
<code>-ssl</code>	Force SSL mode
<code>-o</code>	Output file

Flag	Meaning
<code>-Format</code>	Output format (txt, html, csv, xml)
<code>-Tuning</code>	Restrict to specific test categories
<code>-evasion</code>	IDS evasion techniques
<code>-Display</code>	Control what's shown (e.g., show redirects)
<code>-timeout</code>	Request timeout
<code>-useproxy</code>	Route through a proxy (e.g., Burp for inspection)

```
nikto -h target -o report.html -Format html
nikto -h target -Tuning9 # only run "in
fo disclosure" category
```

4. Tuning Categories (Scoping a Scan)

Nikto's `-Tuning` flag lets you focus on specific check categories instead of running everything:

Code	Category
1	Interesting file/seen in logs
2	Misconfiguration/default file
3	Information disclosure
4	Injection (XSS/script/HTML)
5	Remote file retrieval - inside web root
6	Denial of service
7	Remote file retrieval - server wide
8	Command execution/remote shell
9	SQL injection
x	Reverse tuning (exclude instead of include)

```
nikto -h target -Tuning123 # info disclosure + misconfig
+ interesting files only
nikto -h target -Tuning x6 # everything except DoS checks
```

5. Reading Nikto Output

```
+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-Content-Type-Options header is not set.
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54)
+ OSVDB-3268: /icons/: Directory indexing found.
+ OSVDB-3092: /admin/: This might be interesting...
+ /phpinfo.php: Output from the phpinfo() function was found.
```

Each line is an independent finding — Nikto doesn't prioritize/rank severity the way Nessus/OpenVAS do, so triaging what's actually worth pursuing is a manual judgment call (e.g., a missing security header is low-priority, an exposed `phpinfo.php` leaking full server config is high-priority).

6. Common High-Value Findings

Outdated software versions — feeds directly into the same CVE cross-referencing your SmartScan project already automates.

Directory indexing enabled — ties back to your Google Dorking guide's `intitle:"index of"` technique, just discovered actively instead of passively.

Exposed default/test files (`phpinfo.php`, `test.php`, `/admin/`) — classic forgotten-file findings.

Missing security headers (`X-Frame-Options`, `Content-Security-Policy`, `Strict-Transport-Security`) — low individual severity but worth noting in a report as a hardening recommendation.

7. Nikto's Limitations (Important to Understand)

- **No JavaScript rendering** — can't test modern SPAs (React/Angular/Vue apps) the way a browser-based tool like ZAP can, since it only sees raw HTTP responses
- **High false-positive rate** — Nikto is intentionally broad/noisy; every finding needs manual verification before it goes in a report
- **Very noisy on the wire** — sends a large number of requests quickly, easily detected by any WAF/IDS — not a stealth tool, useful mainly in authorized/whitelisted testing contexts

- **No authentication handling** by default — struggles with anything behind a login without additional configuration

8. Practical Example — Full Workflow

```
# 1. Confirm the web service and grab initial info
nmap -sV -p80,443 target
nc -v target80

# 2. Run Nikto for a broad first-pass
nikto -h target -o nikto_report.html -Format html

# 3. Manually verify anything interesting Nikto flagged
curl http://target/phpinfo.php

# 4. Follow up with deeper tools for anything Nikto can't do
# (directory brute-forcing with gobuster, deeper app testing with ZAP)
gobusterdir -u http://target -w /usr/share/wordlists/dirb/common.txt
```

9. Nikto vs. Other Web Scanners — Where It Fits

Tool	Best For
Nikto	Fast, broad server-level misconfig/known-file checks
Nuclei	Template-based, fast, highly extensible, broader than just web servers
Wapiti	Deeper application-layer vuln testing (injects payloads)
OWASP ZAP	Full interactive/automated application security testing, handles JS-heavy apps

Realistic order in an engagement: **Nikto first** (fast triage) → **Nuclei** (broader template-based sweep) → **ZAP/Wapiti** (deep application testing on anything promising).

10. Kill Chain / ATT&CK Mapping

Framework	Mapping
Cyber Kill Chain	Reconnaissance
MITRE ATT&CK	T1595.002 (Active Scanning: Vulnerability Scanning), T1592.002 (Gather Victim Host Information: Software)

Quick Reference Cheat Sheet

bash

```
nikto -h target
# basic scan
nikto -h target -ssl -p443
# HTTPS
nikto -h target -o report.html -Format html
nikto -h target -Tuning123
# scoped categories
nikto -h target -useproxy http://127.0.0.1:8080
# route through Burp/ZAP
```